Technology Comparison

MathVerity uses Go, PostgreSQL, React with TypeScript and Vite, REST JSON and Docker Compose. The comparison covers five key decisions and assesses alternatives against the requirements of the limited MVP.

Selected approach: Go and PostgreSQL in Docker Compose; React + TypeScript + Vite for the browser; REST JSON for the API. The architecture is a modular monolith with an external AI adapter. Go, PostgreSQL and Docker are developer-specified constraints. The comparison explains their fit to the requirements rather than presenting them as winners of an independent performance benchmark.

Comparison of Five Key Decisions

Table 1. Technology alternatives

Decision	Alternative	Fit for MathVerity	Cost or limitation	Selection
Server	Go and net/http [T01]	Typed domain modules, API and controlled timeouts	Administrative forms and integrations require explicit implementation	Selected
Server	Python and FastAPI [T07]	Convenient for ML experiments and APIs	A second core language is unnecessary without a custom ML model	Not for the core API
Server	Node.js and TypeScript [T07]	Shared frontend language and asynchronous integrations	No benefit that justifies replacing the specified Go stack	Not selected
Data	PostgreSQL [T02]	Parent–child relations, attempt and XP transactions, SQL reports, JSONB	Requires migrations and database operations	Selected
Data	MongoDB [T08]	Flexible content documents	Relational constraints and reporting remain substantial project tasks	Not selected
Data	SQLite [T08]	Simple local database for demos	A client–server database was selected for multiple users and concurrent writes	Not for primary storage

Decision	Alternative	Fit for MathVerity	Cost or limitation	Selection
Frontend	React + TypeScript + Vite [T03, T04]	Reusable items, forms and reports; static build	Routing and request management must be selected	Selected
Frontend	Vue + TypeScript + Vite [T06]	Also suitable for interactive dashboards	Comparable option; reconsider if the team has stronger Vue experience	Acceptable alternative
Frontend	Next.js [T06]	Useful for SSR and a public content site	Private dashboards do not require SSR; adds a server layer	Later for SEO
API	REST JSON [T09]	Few resources, straightforward debugging, OpenAPI	Contracts require disciplined maintenance	Selected
API	GraphQL [T09]	Flexible queries for complex reports	Query complexity controls and permissions add work	Not initially
API	gRPC [T09]	Strict protobuf contracts between services	Browser integration and the lack of microservices reduce its value	After service extraction
Hints	Rules and prepared templates	Predictable and works without a provider	Limited variation	Baseline and fallback
Hints	External LLM through a Go adapter	Rephrases hints using a prepared rubric	Cost, latency, incorrect answers and data processing conditions	After quality evaluation
Hints	Self-hosted open-weight model	Greater infrastructure control	Compute resources and separate operations	Outside the MVP

Table 1 is a qualitative assessment. Technology properties were checked against official documentation [T01–T14]. The selection reflects the project scope and the team’s skills.

Weighted Comparison

To make the selection easier to check, each alternative was scored from 1 (poor) to 5 (strong) against four criteria: fit for the MathVerity requirements (weight 30%); skills of the two-person team (30%); setup and maintenance effort, where a higher score means less effort (20%); and risk and ecosystem maturity,

where a higher score means lower risk (20%). The weighted score is the sum of each score multiplied by its weight.

Table 2. Weighted comparison of alternatives

Decision	Alternative	Fit (30%)	Team skills (30%)	Effort (20%)	Risk (20%)	Weighted score
Server platform	**Go and net/http**	5	5	3	4	**4.40 (selected)**
Server platform	Python and FastAPI	4	2	4	4	3.40
Server platform	Node.js and TypeScript	4	3	4	4	3.70
Data storage	**PostgreSQL**	5	4	3	5	**4.30 (selected)**
Data storage	MongoDB	2	3	4	4	3.10
Data storage	SQLite	3	4	5	3	3.70
Frontend	**React + TypeScript + Vite**	5	4	4	5	**4.50 (selected)**
Frontend	Vue + TypeScript + Vite	5	3	4	4	4.00
Frontend	Next.js	3	3	3	4	3.20
API	**REST JSON**	5	5	4	5	**4.80 (selected)**
API	GraphQL	3	2	2	4	2.70
API	gRPC	2	2	2	4	2.40
Hint generation	**Rules and prepared templates**	4	5	4	5	**4.50 (selected)**
Hint generation	External LLM through a Go adapter	4	3	3	2	3.10
Hint generation	Self-hosted open-weight model	3	1	1	2	1.80

The scores are the team’s own judgement, based on the documentation cited above, and are not benchmark results. Go, PostgreSQL and Docker were given as developer constraints, so the matrix tests whether they remain a reasonable choice and is not an independent contest.

Sensitivity check: when the team-skills criterion is removed and its weight is shared proportionally among the other three, the first-ranked alternative is the same for all five decisions, so the selection does not depend on the team’s familiarity with a tool alone.

Supporting Tools

Go net/http handles HTTP and middleware; pgx is the PostgreSQL driver; sqlc generates typed code from SQL; goose manages SQL migrations [T01, T10–T12]. Initially, pgx may be used without sqlc until queries stabilise. A transaction combines the attempt, diagnostic result and XP award. An idempotency key prevents duplicate awards.

The proposed frontend tools are React Router for screens, TanStack Query for server data, KaTeX with trust disabled and input length limits, and plain CSS or CSS Modules. These libraries are implementation plans; the standalone prototype has no external dependencies. OpenAPI validation and client generation follow agreement on the API contract.

Planned verification includes go test and the race detector for domain logic, integration tests with real PostgreSQL, Vitest for meaningful UI logic, and Playwright for end-to-end scenarios [T13]. Latency and AI error metrics initially use structured logs. OpenTelemetry and Prometheus may follow a working API. Redis is justified only by a measured need for a shared rate limiter or queue. Kubernetes, Kafka, a vector database and a Python service are unnecessary for the initial version.

Version Pinning

Dependency versions and Docker image digests [T05] must be pinned during implementation. The latest tag is not used as a reproducible project version. A local PostgreSQL configuration is not a production deployment.

The local example specifies PostgreSQL 17 with patch version 17.11 [T15]. Availability of the particular Docker tag has not been tested by running it; the digest should be recorded after the first successful image pull.

References

[T01] Go. Accessing relational databases and transactions. https://go.dev/doc/database/ ; https://go.dev/doc/database/execute-transactions

[T02] PostgreSQL. Constraints and JSON types. https://www.postgresql.org/docs/current/ddl-constraints.html ; https://www.postgresql.org/docs/current/datatype-json.html

[T03] React. Quick Start. https://react.dev/learn

[T04] Vite. Getting Started. https://vite.dev/guide/

[T05] Docker Compose. https://docs.docker.com/compose/

[T06] Vue. Introduction. https://vuejs.org/guide/introduction.html ; Next.js documentation: https://nextjs.org/docs

[T07] FastAPI documentation. https://fastapi.tiangolo.com/ ; Node.js: https://nodejs.org/en/learn/getting-started/introduction-to-nodejs

[T08] SQLite. Appropriate Uses. https://www.sqlite.org/whentouse.html ; MongoDB. Data Modeling. https://www.mongodb.com/docs/manual/data-modeling/

[T09] GraphQL. Learn. https://graphql.org/learn/ ; gRPC. Introduction. https://grpc.io/docs/what-is-grpc/introduction/ ; HTTP Semantics: https://www.rfc-editor.org/rfc/rfc9110

[T10] pgx v5 documentation. https://pkg.go.dev/github.com/jackc/pgx/v5

[T11] sqlc documentation. https://docs.sqlc.dev/en/stable/

[T12] goose documentation. https://pressly.github.io/goose/

[T13] KaTeX API. https://katex.org/docs/api ; Playwright. https://playwright.dev/docs/intro

[T14] W3C. Web Content Accessibility Guidelines 2.2. https://www.w3.org/TR/WCAG22/

[T15] PostgreSQL 17.11 release notes, 13 August 2026. https://www.postgresql.org/docs/release/17.11/